

Task 1

1.1 Project Description

York university students interact with eClass across multiple courses every term. Each course contains messy categorization of information like announcements, forum posts, assignment descriptions, quizzes, lecture slides, and links. Students have to manually read through dozens of announcements and reorient themselves to traverse through each course. This becomes time consuming and prone for students to interpret things incorrectly. As a result, students miss deadlines, priorities, and waste time looking for information.

The targeted users are YorkU students enrolled in multiple eClass courses per term. Also, students with accessibility needs who benefit from simple concise summaries, priorities ranking, language interaction, etc.

The AI agent will take in course activity with a live eClass fetch where it grabs info from the site, it summarizes announcements and forums, ranks deadlines and assessments by priority, detects what changed, generates study plans, answers questions about the course. All features are reachable from both GUI and CLI.

The AI layer will be using Qwen3.6 via cloud API and qwen3.5 with Ollama for local/offline runs, with embeddings for RAG over slides and PDFs.

The LLM is gonna be a multi-agent system that interacts with a system that drives the LLM and uses a set of tools like eClass fetcher, PDF parser, etc. There is also a background component that watches for new items between each session.

1.2 Feature Specification

F01 - Course announcement Summarizer (AI-based)

- The user selects courses in GUI and clicks “Summarize Announcements”. The agent retrieves new announcements, analyzes them, and generates a concise summary. The summary is displayed in the GUI and can be expanded or regenerated.

F02 - Deadline & Assessment Summarizer (Hybrid)

- The user opens the “Deadlines” tab and selects a course. The agent reads through course pages, syllabus, and announcements, it then extracts deadlines, weights, and formats, and produces a list. The list is displayed in the GUI and can be edited by the user to correct any errors.

F03 - eClass change detector (Hybrid)

- The user can toggle a “Watch” button on a course. It will then periodically check eClass content when the program is open, or update the eClass content when the program is opened again. The log of new content is displayed in the GUI.

F04 - Task priority ranker (AI-based)

- The user can sort the deadlines tab and rank them with a score. The deadlines tab is then sorted with the new ranked list.

F05 - Progress Tracker (Hybrid)

- The user marks tasks as complete via checkboxes in the GUI. The agent updates progress scores and re prioritises remaining tasks.

F06 - Final grade calculator (Deterministic)

- The user opens the “Grades” tab and enters scores per assessment. The agent computes the total grade and output current average. The user can also have a grade goal where the agent can highlight grades needed per assessment to reach a certain final grade.

F07 - Language Q&A (AI-based)

- The user can type questions in the chat panel. The agent embeds the questions and retrieves relevant chunks from stored course materials, and generates a answer. The answer is displayed in the GUI.

F08 - Study plan generator (AI-based)

- The user can set their availability in the “Study” tab and generate a plan. The agent turns the tasks list into subtasks and estimates the time needed to complete. The plan is displayed in the GUI as a schedule and can be adjusted.

F09 - Slides Summary / Notes (AI-based)

- The user can upload or sync lecture slides and click “notes”. The agent reads the file, extracts key information, and generates structured study notes in a certain style. The notes are displayed in the GUI.

F10 - Weekly eClass report (AI-based)

- The user can click “generate report”. The agent gathers outputs from announcements, deadlines, grades, and puts them in a report. The report is displayed in the GUI and can be exported as a PDF.

F11 - Practice quiz generator (AI-based)

- The user opens the “practice” tab, selects a course, topic, and lecture notes. The agent retrieves relevant materials, generates practice questions, and grades the user’s answers. The quiz and results are displayed in the GUI.

Task 2

Pattern 1 - Template Method

All nine agents follow the same skeleton pattern

1. Validate
2. Fetch data with tools
3. Build prompt
4. Call LLM
5. Parse into AgentResult

The Agent plays the AbstractClass role and defines the template method, which calls the abstract steps in a fixed order. Each concrete agent (AnnouncementAgent, DeadlineAgent, PriorityAgent,...) plays the ConcreteClass role by overriding only the steps it needs.

The nine agents share the same flow but differ in prompts, tools, and parsing. Template method lets me write the flow once and edit each agent's unique features without duplicating the flow across nine classes.

Without it each agent would need to repeat the same process (validate -> fetch -> build -> call -> parse) any change to the flow would need to be made in to all nine agents.

Pattern 2 - Strategy

The system must call an LLM through one of two changeable backends, Qwen3.6 via cloud API or qwen3.5 via Ollama. This is chosen at runtime based on user preference or network availability.

The llmSource plays the Strategy role, CloudQwenSource and LocalQwenSource are ConcreteStrategies, and Agent is the Context that holds a llmSource reference and passes every LLM call to it. The AIFactory at startup decides which concrete strategies to inject.

Strategy is appropriate because both sources do the same job in different ways. Strategy lets the agent code stay identical regardless of which backend is currently active. If the cloud source fails, the agent can switch to the local source at runtime

Without it every agent would need to check what LLM is active, and adding another model would need to update all nine agents.

Pattern 3 - Observer

When new eClass content arrives, a task is marked done, or a new deadline is added, several parts of the system need to react at once.

1. Priority ranker re sorts remaining tasks
2. Progress view recomputes
3. GUI refreshes

These must happen without the domain objects knowing the UI.

Subject is the interface with adding and removing observers. Observer is the interface that updates. Course plays the ConcreteSubject which holds the list of observers and notifies

whenever its content changes. ProgressDisplay is a ConcreteObserver that reacts by re-rendering the progress panel. PriorityRanker is another that re-sorts deadlines.

Observer is appropriate because one event has multiple parts reacting to it. Observer removes the Course from the reactions, so new reactions can be added without editing Course.

Without it Course would need to direct references to ProgressDisplay, PriorityRanker, and any future listener. Adding a new reactor would require changing the domain (Course) class everytime

Pattern 4 - Factory

My project must run in 2 modes, online (cloud LLM) and offline (local Ollama). Each mode requires AI components, mixing cloud LLM with a local vector store would break the embedding compatibility.

AIFactory uses AbstractFactory design declaring createLlmSource(), createVectorStore(), and createEmbedder(). OnlineAIFactory and OfflineAIFactory are ConcreteFactories. Agent controller receives an AIFactory at startup and uses it.

The offline and online mode affects three components The Abstract Factory guarantees the family is created the same, so no code can produce intertwined data.

Without it the code could mix local and cloud data.

Pattern 5 - Facade

AppFacade has one method per feature and hides the entire internal system behind a small interface. The system hides eight controllers.

The clients (GUI or CLI) can only call AppFacade.

This is appropriate because all features follow the same flow, and the internal system is complex with multi-agent pipelines, tool registry, LLM factory. Facade shows a single entry point per feature so the UI never has to know about agents, tools, or factories.

Without it the GUI and CLI would need to import and coordinate the controller agents. Any internal changes would have an effect in the front-end.

Task 3

Feature	Description	Type	Related Use Case	Classes	Key Methods	Sequence Diagram	Design Pattern(s)
F01	Course announcement summarizer (AI-based)	AI	UC01 Summarize Announcements	SummarizeAnnouncementsCommand, AppFacade, AnnouncementController, MasterAgent, AnnouncementAgent, EclassFetcherTool, ILImSource, course, ProgressDisplay		seqf01	Template, Facade, Strategy, Abstract Factory, Observer
F02	Deadline & assessment summarizer (Hybrid)	Hybrid	UC02 View & Rank Deadlines	DeadlineCommand, AppFacade, DeadlineController, MasterAgent, DeadlineAgent, EClassFetcherTool, PDFParserTool, ILImSource		seqf02	Facade, Strategy, Abstract Factory, Template
F03	eClass change detector (Hybrid)	Hybrid		DeadlineController, MasterAgent, DeadlineAgent, EClassFetcherTool			Observer, Facade
F04	Task priority ranker (AI-based)	AI	UC02 View & Rank Deadlines	DeadlineCommand, AppFacade, DeadlineController, MasterAgent, PriorityAgent, ILImSource, Deadline		seqf04	Facade, Strategy, Observer, Template
F05	Progress tracker (Hybrid)	Hybrid	UC04 Track Progress	DeadlineCommand, AppFacade, DeadlineController, Deadline, Course, Observer		seqf05	Facade, Observer
F06	Final grade calculator (Deterministic)	Deterministic	UC05 Calculate Final Grade	GradesCommand, AppFacade, GradesController, GradeCalculator, Grade, GradeReport, Assessment		seqf06	Facade
F07	Language Q&A (AI-based, RAG)	AI	UC06 Ask Course Q&A	ChatCommand, AppFacade, ChatController, MasterAgent,		seqf07	Facade, Strategy, Abstract Factory ,

				ChatAgent, EmbeddingTool, ILlmSource, CourseMaterial			Template
F08	Study plan generator (AI-based)	AI	UC07 Generate Study Plan	StudyCommand, AppFacade, StudyController, MasterAgent, StudyPlanAgent, ILlmSource, Deadline, Assessment		seqf08	Facade, Strategy, Template
F09	Slides summary / notes (AI-based)	AI	UC08 Generate Notes from Slides	StudyCommand, AppFacade, StudyController, MasterAgent, NotesAgent, PDFParserTool, EmbeddingTool, ILlmSource, CourseMaterial		seqf09	Facade, Strategy, Abstract Factory, Template
F10	Weekly eClass report (AI-based)	AI	UC09 Generate Weekly Report	ReportCommand, AppFacade, ReportController, MasterAgent, ReportAgent, AnnouncementAgent, DeadlineAgent, GradesController, ILlmSource		seqf10	Facade, Strategy, Abstract Factory, Template
F11	Practice quiz generator (AI-based)	AI	UC10 Take Practice Quiz	QuizCommand, AppFacade, QuizController, MasterAgent, QuizAgent, EmbeddingTool, ILlmSource, CourseMaterial		seqf11	Facade, Strategy, Abstract Factory, Template